AI TECHNICAL DEBT PRIORITIZER | SUMMARY 1 OF 3

Core ML, AI and Algorithms

How the system learns, ranks, explains and plans technical-debt fixes (short version of design document 1)

1. Problem

TDP ranks the source files of a repository by how much maintenance pain each is likely to cause in the next 12 months, explains why, and turns the ranking into a fix plan that fits a time budget. The unit is a **file at a snapshot time T**. Every feature uses only history up to T; every label uses only what happens in **(T, T + 12 months]**. That separation is what makes it a real prediction instead of a description of the past.



Pipeline. Amber (labels) is used only for training and evaluation.

2. Dataset

- 20 to 30 public Python repositories: at least 3 years and 1,000 commits, 80 or more source files, bug-labelled issues, recent activity, permissive license, diverse domains.
- Snapshots every 6 months; one row per **(repository, snapshot, file)**, about 45,000 rows and 150 ranking queries.
- Renames are followed; merge commits and bulk commits (more than 50 files) are excluded from churn and coupling.

3. Features (about 40, plus within-repository percentiles)

Family	Key features	Reveals
S Static (Tree-sitter)	SLOC, cyclomatic complexity (sum, mean, max), nesting, duplication ratio (token winnowing)	complexity, clones
E Evolution (git)	recency-decayed churn (half-life 90 days), relative churn, commit counts, age, complexity trend	change hotspots
O Ownership	owner share, minor contributors (under 5%), author entropy, owner still active	knowledge debt
D Defect history	bug-fix commits and SZZ-traced bugs **before** T	chronic defect sites
I Impact (NetworkX)	fan-in/out, transitive dependents, PageRank, betweenness, cycle size, hidden co-change coupling	blast radius, architecture
T Tests (static)	matching test file, test files importing it, test co-change rate, package test ratio	test debt
Q SATD	count, density and age of self-admitted debt comments (TODO, HACK, ...)	admitted debt

4. Labels

- **Bug-fix commits:** issue links with a bug label first, then filtered keywords; precision checked on 200 hand-labelled commits.
- **SZZ:** take the lines a fix changed, run git blame on its parent, find the commit that introduced them.
- **Latent defect:** a fix in (T, T+H] repairs lines in the file that already existed at T.
- **Pain** P = defects + 0.25 · log(1 + churn / repo median churn), converted to grades 0 to 4 by within-repository quantiles. A sensitivity analysis varies the weight and the horizon.

5. Model

- **XGBoost LambdaMART** (objective rank:ndcg), one query per (repository, snapshot). It optimises NDCG@k = $\sum (2^{rel} - 1) / \log_2(i + 1)$, normalised by the ideal ordering, so mistakes at the top cost most.
- Optuna tuning on time-ordered, grouped folds; optional monotone constraints (more churn never lowers priority).
- **Baselines:** random, LOC, complexity, churn x complexity hotspot, prior bugs, Random Forest, pointwise XGBoost.

6. Evaluation

- **Temporal split** with an embargo (no training label window crosses the test snapshot) and **leave-one-project-out** (every test repository is unseen).
- **Metrics:** NDCG@10, Precision@10, Recall@20% of LOC, P_{opt} (effort-aware lift), Spearman, Brier score and calibration error.
- **Ablation** by feature family (add one, remove one); Wilcoxon tests, Cliff's delta, bootstrap confidence intervals.
- Leakage and label-shuffle tests; PROMISE used only as a sanity check for the static extractors.

7. Scores and explanations

- **Priority** = percentile within the repository (92 means above 92% of its files). Bands: Critical 90+, High 75 to 89, Medium 50 to 74, Low below 50.

- **Risk** = isotonic-calibrated probability of a latent defect being fixed within 12 months.
- **TreeSHAP** values summed per feature family, then written as sentences, e.g. "changes very often: 47 commits in 6 months (top 4% in repo)". Each file also gets a **debt type**: complexity, duplication, test, architecture, knowledge, admitted or defect hotspot.

8. Fix plan

- benefit = risk · $(1 + \log_2(1 + \text{transitive dependents}))$; cost = SQALE-style remediation hours, shown as a range.
- **0/1 knapsack** by dynamic programming, $O(n \cdot B)$ in half-hour units, maximises benefit within the budget (greedy ratio fallback, at least half of optimal).
- **Order:** collapse dependency cycles (Tarjan SCC), topological sort so foundations come first, ties broken by ROI.

9. AI layer

- **SATD classifier:** TF-IDF + logistic regression trained on about 62,000 labelled comments.
- **LLM advisor** (on request, top files only): gets reasons, debt type and the most complex functions, returns JSON; schema-validated, every function name checked against the syntax tree, shown as advice. Scored with a rubric and a hallucination rate.
- **What-if simulator** (model counterfactual, not causal) and **feedback** as pairwise preferences with active learning.